

## Frequency Array

Instead of searching again & again, we can use frequency array for,

- \* Find duplicate.
- \* Count frequency.
- \* Check If pair exist.
- \* If value already appear
- \* To store the occurrence.

Note :-

we can use frequency array when our value is small.  
when our range values are high we need to use hashing.

## 1748. Sum of Unique Elements

**Input:** nums = [1,2,3,2]

**Output:** 4

**Explanation:** The unique elements are [1,3], and the sum is 4.

st1: create a freq array for 1 to 100.

```
int freq[] = new int [101];
```

Step 2: Array values & freq index values should be same.

[illegible]

```
int val = arr[i];  
val = 1;  
freq[val]++;
```

```
public int sumOfUnique(int[] nums) {
    int freq[] = new int[101];
    for(int i=0;i<nums.length;i++){
        int val = nums[i];
        freq[val]++;
    }
    int sum = 0;
    for(int i=0;i<101;i++){
        if(freq[i]==1){
            sum+=i;
        }
    }
    return sum;
}
```

## 1832. Check if the Sentence Is Pangram

**Input:** sentence = "thequickbrownfoxjumpsoverthelazydog"

**Output:** true

**Explanation:** sentence contains at least one of every letter of the English alphabet.

|   |   |   |   |   |   |   |   |   |   |    |    |    |    |    |    |    |    |    |    |    |    |    |    |    |    |
|---|---|---|---|---|---|---|---|---|---|----|----|----|----|----|----|----|----|----|----|----|----|----|----|----|----|
| a | b | c | d | e | f | g | h | i | j | k  | l  | m  | n  | o  | p  | q  | r  | s  | t  | u  | v  | w  | x  | y  | z  |
| 1 | 1 | 1 | 1 | 3 | 1 | 1 | 2 | 1 | 1 | 1  | 1  | 1  | 1  | 4  | 1  | 1  | 2  | 1  | 2  | 2  | 1  | 1  | 1  | 1  | 1  |
| 0 | 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 | 9 | 10 | 11 | 12 | 13 | 14 | 15 | 16 | 17 | 18 | 19 | 20 | 21 | 22 | 23 | 24 | 25 |

$ch: 'd' \Rightarrow ch - 'a' = 100 - 97 = 3.$

```
public boolean checkIfPangram(String sentence) {
    int freq[] = new int[26];
    for(int i=0;i<sentence.length();i++){
        char ch = sentence.charAt(i);
        freq[ch-'a']++;
    }
    for(int i=0;i<26;i++){
        if(freq[i]==0) return false;
    }
    return true;
}
```

## 2716. Minimize String Length

**Input:** s = "aaabc"

**Output:** 3

**Explanation:**

- Operation 2: we choose  $i = 1$  so  $c$  is 'a', then we remove  $s[2]$  as it is closest 'a' character to the right of  $s[1]$ .  
s becomes "aabc" after this.
- Operation 1: we choose  $i = 1$  so  $c$  is 'a', then we remove  $s[0]$  as it is closest 'a' character to the left of  $s[1]$ .  
s becomes "abc" after this.

Approach 1:-

```
public int minimizedStringLength(String s) {
    int freq[] = new int[26];
    for(int i=0;i<s.length();i++){
        char ch = s.charAt(i);
        freq[ch-'a']++;
    }
    int cnt = 0;
    for(int i=0;i<26;i++){
        if(freq[i]>0){
            cnt++;
        }
    }
    return cnt;
}
```

Approach 2:-

```
public int minimizedStringLength(String s) {
    int freq[] = new int[26];
    for(int i=0;i<s.length();i++){
        char ch = s.charAt(i);
        freq[ch-'a']=1;
    }
    int cnt = 0;
    for(int i=0;i<26;i++){
        cnt+=freq[i];
    }
    return cnt;
}
```

### 3120. Count the Number of Special Characters I

**Input:** word = "aaAbcBC"

**Output:** 3

**Explanation:**

The special characters in word are 'a', 'b', and 'c'.

```
public int numberOfSpecialChars(String word) {
    int freq1[] = new int[26]; //Upper case
    int freq2[] = new int[26]; //lower case
    for (int i = 0; i < word.length(); i++) {
        char ch = word.charAt(i);
        if (Character.isUpperCase(ch)) {
            freq1[ch - 'A']++;
        } else {
            freq2[ch - 'a']++;
        }
    }
    int cnt = 0;
    for (int i = 0; i < 26; i++) {
        if (freq1[i] > 0 && freq2[i] > 0) {
            cnt++;
        }
    }
    return cnt;
}
```

### 1941. Check if All Characters Have Equal Number of Occurrence:

**Input:** s = "abacbc"

**Output:** true

**Explanation:** The characters that appear in s are 'a', 'b', and 'c'. All characters occur 2 times in s.

```
public boolean areOccurrencesEqual(String s) {
    int freq [] = new int[26];
    for(int i=0;i<s.length();i++){
        char ch = s.charAt(i);
        freq[ch-'a']++;
    }
    char ch1 = s.charAt(0);
    int freqcnt = freq[ch1-'a']; //2
    for(int i=0;i<26;i++){
        if(freq[i]!=0 && freqcnt != freq[i]){
            return false;
        }
    }
    return true;
}
```

### 3005. Count Elements With Maximum Frequency

**Input:** nums = [1,2,2,3,1,4]

**Output:** 4

**Explanation:** The elements 1 and 2 have a frequency of 2 which is the maximum frequency in the array.

So the number of elements in the array with maximum frequency is 4.

```
public int maxFrequencyElements(int[] nums) {
    int freq[] = new int[101];
    for(int i=0;i<nums.length;i++){
        int val = nums[i];
        freq[val]++;
    }
    int max = 0;
    for(int i=0;i<101;i++){
        max = Math.max(max,freq[i]);
    }

    int sum = 0;
    for(int i=0;i<101;i++){
        if(freq[i]==max){
            sum += max;
        }
    }
    return sum;
}
```

### 2351. First Letter to Appear Twice

**Input:** s = "abccbaacz"

**Output:** "c"

**Explanation:**

The letter 'a' appears on the indexes 0, 5 and 6.

The letter 'b' appears on the indexes 1 and 4.

The letter 'c' appears on the indexes 2, 3 and 7.

The letter 'z' appears on the index 8.

The letter 'c' is the first letter to appear twice, because out of all the letters the index of its second occurrence is the smallest.

```
public char repeatedCharacter(String s) {
    int freq[] = new int[26];
    for(int i=0;i<s.length();i++){
        char ch = s.charAt(i);
        if(freq[ch-'a']!=0){
            return ch;
        }else{
            freq[ch-'a']++;
        }
    }
    return 'b';
}
```

| a | b | c | d | e |
|---|---|---|---|---|
| 1 | 1 | 2 |   |   |

## Searching.



### Linear Search:-

ip:-

|   |   |   |   |   |   |
|---|---|---|---|---|---|
| 2 | 8 | 7 | 5 | 6 | 9 |
|---|---|---|---|---|---|

target = 5.

```
public static boolean LinearSearch(int arr[], int target){
    for(int i=0; i<arr.length; i++){
        if(arr[i]==target){
            return true;
        }
    }
    return false;
}
```

### Binary Search.

Note:- Array should be in sorted order. (Ascending order)

ip:-

|   |   |   |   |   |    |    |    |
|---|---|---|---|---|----|----|----|
| 1 | 4 | 5 | 7 | 9 | 11 | 12 | 15 |
|---|---|---|---|---|----|----|----|

target = 12.

op:- 6 → Index.

|   |   |   |   |   |    |    |    |
|---|---|---|---|---|----|----|----|
| 1 | 4 | 5 | 7 | 9 | 11 | 12 | 15 |
|---|---|---|---|---|----|----|----|

Declare st & end position st=0 end=len-1.

mid =

| st | mid. |   |   |   |    | end |    |
|----|------|---|---|---|----|-----|----|
| 1  | 4    | 5 | 7 | 9 | 11 | 12  | 15 |

st=0 end=7.

mid = st + (end - st) / 2.

= 0 + (7 - 0) / 2 = 3

| st | mid. |   |   |   |    | end |    |
|----|------|---|---|---|----|-----|----|
| 1  | 4    | 5 | 7 | 9 | 11 | 12  | 15 |

If mid val == target.

check target is greater than or less than mid val.

12 → greater than mid val.

|   |   |   | st | mid |    |    | end |
|---|---|---|----|-----|----|----|-----|
| 1 | 4 | 5 | 7  | 9   | 11 | 12 | 15  |

st = 4      end = 7.

$$4 + (7 - 4) / 2 = 4 + 1 = 5$$

```
public static int BinarySearch(int arr[], int target){
    int st = 0, end = arr.length-1;
    while(st<=end){
        int mid = st+(end-st)/2;
        if(target==arr[mid]){
            return mid;
        }else if(target<arr[mid]){
            end = mid-1;
        }
        else{
            st = mid+1;
        }
    }
    return -1;
}
```

If target is greater than mid. update start.

st = mid+1;

If target is lesser than mid. update end

end = mid-1;

## 704. Binary Search

Input: nums = [-1,0,3,5,9,12], target = 9

Output: 4

Explanation: 9 exists in nums and its index is 4

```
public int search(int[] arr, int target) {
    int st = 0, end = arr.length-1;
    while(st<=end){
        int mid = st+(end-st)/2;
        if(target==arr[mid]){
            return mid;
        }else if(target<arr[mid]){
            end = mid-1;
        }
        else{
            st = mid+1;
        }
    }
    return -1;
}
```

## 35. Search Insert Position

Input: nums = [1,3,5,6], target = 5

Output: 2

```
public int searchInsert(int[] arr, int target) {
    int st = 0, end = arr.length-1;
    while(st<=end){
        int mid = st+(end-st)/2;
        if(target==arr[mid]){
            return mid;
        }else if(target<arr[mid]){
            end = mid-1;
        }
        else{
            st = mid+1;
        }
    }
    return st;
}
```

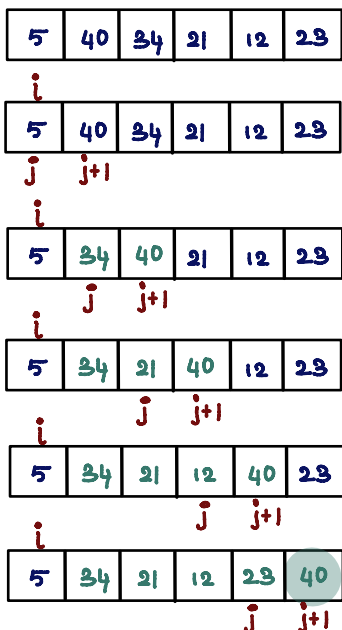
## Sorting.

It is used to sort the array values in ascending order.

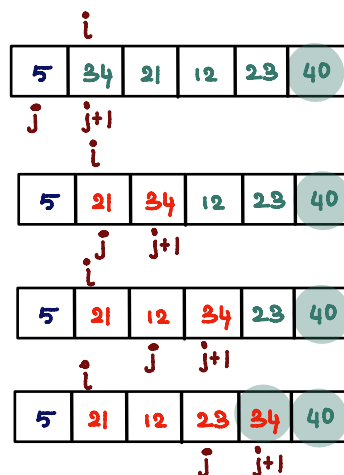
Types:-

- \* Bubble Sort.
- \* Selection Sort.
- \* Insertion Sort.

### Bubble Sort :-



No of check = 5



No of check = 4

Why  $-i$ ?

$$n = 6 - 1 = 5 - 1$$

$$\text{No of check} = 4.$$

$$n = 6 - 1 = 5 - 2 = 3.$$

```

public static void BubbleSort(int arr[]){
    int n = arr.length;
    for(int i=0;i<n;i++){
        for(int j=0;j<n-1-i;j++){
            if(arr[j]>arr[j+1]){
                int temp = arr[j];
                arr[j] = arr[j+1];
                arr[j+1] = temp;
            }
        }
    }
    for(int i=0;i<n;i++){
        System.out.print(arr[i]+" ");
    }
}

```

for loop :-  $i=0$        $i < n$   
 $j=0$        $j < n-1-i$

Swap:-  $arr[j] > arr[j+1]$  .  
 $int\ t = arr[j];$   
 $arr[j] = arr[j+1];$   
 $arr[j+1] = t;$

### 905. Sort Array By Parity

**Input:** nums = [3,1,2,4]

**Output:** [2,4,3,1]

**Explanation:** The outputs [4,2,3,1], [2,4,1,3], and [4,2,1,3] would also be accepted.

```

public int[] sortArrayByParity(int[] nums) {
    int n=nums.length;
    for(int i=0;i<n;i++){
        for(int j=0;j<n-i-1;j++){
            if(nums[j]%2!=0 && nums[j+1]%2==0){
                int temp=nums[j];
                nums[j]=nums[j+1];
                nums[j+1]=temp;
            }
        }
    }
    return nums;
}

```

### 922. Sort Array By Parity II

**Input:** nums = [4,2,5,7]

**Output:** [4,5,2,7]

**Explanation:** [4,7,2,5], [2,5,4,7], [2,7,4,5] would also have been accepted.

```
public int[] sortByParityII(int[] nums) {
    int n = nums.length;
    for (int i = 0; i < nums.length; i++) {
        if (i % 2 == 0 && nums[i] % 2 != 0) { // even idx, odd val
            for (int j = i + 1; j < n; j++) {
                if (nums[j] % 2 == 0) { // even val
                    int temp = nums[j];
                    nums[j] = nums[i];
                    nums[i] = temp;
                    break;
                }
            }
        } else if (i % 2 != 0 && nums[i] % 2 == 0) { // odd idx, even val
            for (int j = i + 1; j < n; j++) {
                if (nums[j] % 2 != 0) { // odd val
                    int temp = nums[j];
                    nums[j] = nums[i];
                    nums[i] = temp;
                    break;
                }
            }
        }
    }
    return nums;
}
```

---